

Machine Learning Regression and Image Classification Project

1. Title Page

Project Title: Machine Learning Regression and Image Classification Project

Course: _____

Faculty: _____

Team Number: _____

Submission Date: 2026-05-09

2. Introduction

This project presents a complete supervised and unsupervised machine learning workflow using only the two datasets provided by the student. The first task is a numerical regression problem using a cyber security salary dataset. The second task is an image classification problem using a flower image dataset organized in class folders.

The project is designed for academic discussion. It separates preprocessing, feature engineering, training, evaluation, reporting, and saved model artifacts into clear Python modules. The code automatically inspects the provided datasets and uses one central configuration file for paths, target variables, class selection, image size, and model hyperparameters.

3. Objectives

- Build two regression models for a numerical dataset: Linear Regression and KNN Regressor.
- Build two image models with a maximum of five classes: Logistic Regression classifier and KMeans clustering-based classifier.
- Apply correct preprocessing for numerical and image datasets.
- Extract explainable features and document their dimensions.
- Evaluate regression using MAE, MSE, RMSE, and R2.
- Evaluate classification using accuracy, confusion matrix, and classification report.
- Save trained models, plots, reports, Word documents, and a PDF report.

4. Dataset 1: Numerical Dataset

Dataset name: Cyber Security Salaries Dataset

Source: User-provided CSV file placed in datasets/numerical/

Dataset file: C:\Users\youuss\Documents\machine learning\ML_Project\datasets\numerical\salaries_cyber.csv

Total samples used: 1247

Original shape: [1247, 11]

Used shape after removing missing target rows and configured leakage columns: [1247, 9]

Target variable: salary_in_usd

Raw input columns: work_year, experience_level, employment_type, job_title, employee_residence, remote_ratio, company_location, company_size

Dropped columns: salary, salary_currency

Missing value summary: {'work_year': 0, 'experience_level': 0, 'employment_type': 0, 'job_title': 0, 'salary': 0, 'salary_currency': 0, 'salary_in_usd': 0, 'employee_residence': 0, 'remote_ratio': 0, 'company_location': 0, 'company_size': 0}

Target skewness before training: 2.4795

Target transform decision: Target is strongly skewed and eligible for log1p testing.

Number of extracted/encoded features: 206

Train/validation/test split:

- Training samples: 872
- Validation samples: 187
- Testing samples: 188

5. Dataset 2: Image Dataset

Dataset name: Flowers Recognition Image Dataset

Source: User-provided image folders placed in datasets/images/

Number of classes: 5

Class labels: daisy, dandelion, rose, sunflower, tulip

Samples per class: {'daisy': 764, 'dandelion': 1052, 'rose': 784, 'sunflower': 733, 'tulip': 984}

Total image samples used: 4317

Image size used by the implementation: 128x128 pixels

Train/validation/test split:

- Training samples: 3021
- Validation samples: 648
- Testing samples: 648

6. Preprocessing

For the numerical dataset, missing numerical values are imputed with the median, missing categorical values are imputed with the most frequent category, categorical features are one-hot encoded, and numerical values are standardized using StandardScaler. A conservative IQR-based capper is applied to numerical inputs before scaling to reduce the effect of extreme values without discarding many rows. The configured target column is excluded from the input matrix. The columns `salary` and `salary\_currency` are removed because `salary\_in\_usd` is the selected target and these fields could create target leakage.

The target distribution is inspected before training. If the target is strongly skewed and a log transform improves validation RMSE, log1p is used through a transformed regressor. If it does not improve validation RMSE, the original target scale is kept.

For the image dataset, images are loaded from class folders under `datasets/images/`, converted to RGB, resized to 128x128, normalized to the range 0 to 1, and split using stratification so that each class is represented in training, validation, and testing. Image feature scaling is handled with StandardScaler, and PCA is used only when it improves validation performance or training stability.

7. Feature Extraction

Numerical feature extraction:

- Original numerical columns: work_year, remote_ratio
- Original categorical columns: experience_level, employment_type, job_title, employee_residence, company_location, company_size
- Encoding method: one-hot encoding for categorical variables
- Numerical outlier handling: IQR-based capping on the training split, then reuse of learned bounds on validation and test data
- Scaling method: StandardScaler for numerical variables
- Number of extracted/encoded features: 206
- Final feature dimensions: train [872, 198], validation [187, 198], test [188, 198]
- Feature names: work_year, remote_ratio, experience_level_EN, experience_level_EX, experience_level_MI, experience_level_SE, employment_type_CT, employment_type_FL, employment_type_FT, employment_type_PT, job_title_Application Security Analyst, job_title_Application Security Architect, job_title_Application Security Engineer, job_title_Application Security Specialist, job_title_Azure Security Engineer, job_title_Chief Information Security Officer, job_title_Cloud Security Architect, job_title_Cloud Security Engineer, job_title_Cloud Security Engineering Manager, job_title_Computer Forensic Software Engineer, job_title_Concierge Security Engineer, job_title_Corporate Infrastructure Security Engineer, job_title_Corporate Security Engineer, job_title_Cyber Program Manager, job_title_Cyber Security Analyst, job_title_Cyber Security Architect, job_title_Cyber Security Consultant, job_title_Cyber Security Engineer, job_title_Cyber Security Researcher, job_title_Cyber Security Specialist, job_title_Cyber Security Training Specialist, job_title_Cyber Threat Analyst, job_title_Cyber Threat Intelligence Analyst, job_title_Data Security Analyst, job_title_Detection Engineer, job_title_DevOps Security Engineer, job_title_DevSecOps Engineer, job_title_Digital Forensics Analyst, job_title_Director of Information Security, job_title_Enterprise Security Engineer, job_title_Ethical Hacker, job_title_Head of Information Security, job_title_Head of Security, job_title_IAM Engineer, job_title_IT Security Analyst, job_title_IT Security Engineer, job_title_IT Security Manager, job_title_Incident Response Analyst, job_title_Incident Response Lead, job_title_Incident Response Manager, job_title_Information Security Analyst, job_title_Information Security Architect, job_title_Information Security Compliance Analyst, job_title_Information Security Compliance Lead, job_title_Information Security Compliance Manager, job_title_Information Security Engineer, job_title_Information Security Manager, job_title_Information Security Officer, job_title_Information Security Specialist, job_title_Information Systems Security Engineer, job_title_Infrastructure Security Engineer, job_title_Lead Application Security Engineer, job_title_Lead Information Security Engineer, job_title_Lead Security Engineer, job_title_Network Security Engineer, job_title_Network and Security Engineer, job_title_Offensive Security Engineer, job_title_Penetration Tester, job_title_Penetration Testing Engineer, job_title_Principal Cloud Security Engineer,

job_title_Principal Security Engineer, job_title_Privacy Manager, job_title_Product Security Engineer, job_title_SOC Analyst, job_title_Security Analyst, job_title_Security Consultant, job_title_Security DevOps Engineer, job_title_Security Engineer, job_title_Security Engineering Manager, job_title_Security Incident Response Engineer, job_title_Security Officer, job_title_Security Officer 3, job_title_Security Operations Analyst, job_title_Security Operations Engineer, job_title_Security Researcher, job_title_Security Specialist, job_title_Software Security Engineer, job_title_Staff Application Security Engineer, job_title_Staff Security Engineer, job_title_Threat Hunting Lead, job_title_Threat Intelligence Analyst, job_title_Threat Intelligence Response Analyst, job_title_Vulnerability Analyst, job_title_Vulnerability Management Engineer, job_title_Vulnerability Researcher, employee_residence_AE, employee_residence_AF, employee_residence_AR, employee_residence_AT, employee_residence_AU, employee_residence_AZ, employee_residence_BE, employee_residence_BG, employee_residence_BR, employee_residence_BW, employee_residence_CA, employee_residence_CH, employee_residence_CL, employee_residence_CZ, employee_residence_DE, employee_residence_DK, employee_residence_DZ, employee_residence_EE, employee_residence_EG, employee_residence_ES, employee_residence_ET, employee_residence_FR, employee_residence_GB, employee_residence_GH, employee_residence_GR, employee_residence_HU, employee_residence_ID, employee_residence_IE, employee_residence_IL, employee_residence_IN, employee_residence_IR, employee_residence_IT, employee_residence_JP, employee_residence_KE, employee_residence_LT, employee_residence_LU, employee_residence_MX, employee_residence_NG, employee_residence_NL, employee_residence_NO, employee_residence_NZ, employee_residence_PK, employee_residence_PL, employee_residence_PT, employee_residence_RO, employee_residence_RU, employee_residence_SA, employee_residence_SE, employee_residence_SG, employee_residence_SI, employee_residence_TR, employee_residence_TW, employee_residence_US, employee_residence_VN, employee_residence_ZA, company_location_AE, company_location_AQ, company_location_AR, company_location_AT, company_location_AU, company_location_AX, company_location_AZ, company_location_BE, company_location_BR, company_location_BW, company_location_CA, company_location_CH, company_location_CL, company_location_CZ, company_location_DE, company_location_DK, company_location_DZ, company_location_EE, company_location_EG, company_location_ES, company_location_ET, company_location_FR, company_location_GB, company_location_GR, company_location_HU, company_location_ID, company_location_IE, company_location_IL, company_location_IN, company_location_IT, company_location_JP, company_location_KE, company_location_LU, company_location_MX, company_location_NL, company_location_NO, company_location_NZ, company_location_PK, company_location_PL, company_location_PT, company_location_RO, company_location_RS, company_location_RU, company_location_SA, company_location_SE, company_location_SG, company_location_SI, company_location_TR, company_location_TW, company_location_UM, company_location_US, company_location_VN, company_location_ZA, company_size_L, company_size_M, company_size_S

Image feature extraction:

- RGB color histograms: 16 bins per channel, producing 48 features
- Grayscale histogram: 32 bins
- RGB channel statistics: mean, standard deviation, minimum, and maximum for each channel, producing 12 features
- Grayscale and texture statistics: grayscale mean, grayscale standard deviation, gradient mean, gradient standard deviation, and edge density, producing 5 features
- Local color summary: a 4x4 spatial grid with mean and standard deviation for each RGB channel
- HOG-style gradient orientation features on grayscale image patches
- Total extracted image features: 769
- Final feature dimensions: train [3021, 769], validation [648, 769], test [648, 769]
- Feature names: red_hist_bin_0, red_hist_bin_1, red_hist_bin_2, red_hist_bin_3, red_hist_bin_4, red_hist_bin_5, red_hist_bin_6, red_hist_bin_7, red_hist_bin_8, red_hist_bin_9, red_hist_bin_10, red_hist_bin_11, red_hist_bin_12, red_hist_bin_13, red_hist_bin_14, red_hist_bin_15, green_hist_bin_0, green_hist_bin_1, green_hist_bin_2, green_hist_bin_3, green_hist_bin_4, green_hist_bin_5, green_hist_bin_6, green_hist_bin_7, green_hist_bin_8, green_hist_bin_9, green_hist_bin_10, green_hist_bin_11, green_hist_bin_12, green_hist_bin_13, green_hist_bin_14, green_hist_bin_15, blue_hist_bin_0, blue_hist_bin_1, blue_hist_bin_2, blue_hist_bin_3, blue_hist_bin_4, blue_hist_bin_5,

[illegible]

[illegible]

hog_cell_6_1_bin_1, hog_cell_6_1_bin_2, hog_cell_6_1_bin_3, hog_cell_6_1_bin_4, hog_cell_6_1_bin_5,
hog_cell_6_1_bin_6, hog_cell_6_1_bin_7, hog_cell_6_1_bin_8, hog_cell_6_2_bin_0, hog_cell_6_2_bin_1,
hog_cell_6_2_bin_2, hog_cell_6_2_bin_3, hog_cell_6_2_bin_4, hog_cell_6_2_bin_5, hog_cell_6_2_bin_6,
hog_cell_6_2_bin_7, hog_cell_6_2_bin_8, hog_cell_6_3_bin_0, hog_cell_6_3_bin_1, hog_cell_6_3_bin_2,
hog_cell_6_3_bin_3, hog_cell_6_3_bin_4, hog_cell_6_3_bin_5, hog_cell_6_3_bin_6, hog_cell_6_3_bin_7,
hog_cell_6_3_bin_8, hog_cell_6_4_bin_0, hog_cell_6_4_bin_1, hog_cell_6_4_bin_2, hog_cell_6_4_bin_3,
hog_cell_6_4_bin_4, hog_cell_6_4_bin_5, hog_cell_6_4_bin_6, hog_cell_6_4_bin_7, hog_cell_6_4_bin_8,
hog_cell_6_5_bin_0, hog_cell_6_5_bin_1, hog_cell_6_5_bin_2, hog_cell_6_5_bin_3, hog_cell_6_5_bin_4,
hog_cell_6_5_bin_5, hog_cell_6_5_bin_6, hog_cell_6_5_bin_7, hog_cell_6_5_bin_8, hog_cell_6_6_bin_0,
hog_cell_6_6_bin_1, hog_cell_6_6_bin_2, hog_cell_6_6_bin_3, hog_cell_6_6_bin_4, hog_cell_6_6_bin_5,
hog_cell_6_6_bin_6, hog_cell_6_6_bin_7, hog_cell_6_6_bin_8, hog_cell_6_7_bin_0, hog_cell_6_7_bin_1,
hog_cell_6_7_bin_2, hog_cell_6_7_bin_3, hog_cell_6_7_bin_4, hog_cell_6_7_bin_5, hog_cell_6_7_bin_6,
hog_cell_6_7_bin_7, hog_cell_6_7_bin_8, hog_cell_7_0_bin_0, hog_cell_7_0_bin_1, hog_cell_7_0_bin_2,
hog_cell_7_0_bin_3, hog_cell_7_0_bin_4, hog_cell_7_0_bin_5, hog_cell_7_0_bin_6, hog_cell_7_0_bin_7,
hog_cell_7_0_bin_8, hog_cell_7_1_bin_0, hog_cell_7_1_bin_1, hog_cell_7_1_bin_2, hog_cell_7_1_bin_3,
hog_cell_7_1_bin_4, hog_cell_7_1_bin_5, hog_cell_7_1_bin_6, hog_cell_7_1_bin_7, hog_cell_7_1_bin_8,
hog_cell_7_2_bin_0, hog_cell_7_2_bin_1, hog_cell_7_2_bin_2, hog_cell_7_2_bin_3, hog_cell_7_2_bin_4,
hog_cell_7_2_bin_5, hog_cell_7_2_bin_6, hog_cell_7_2_bin_7, hog_cell_7_2_bin_8, hog_cell_7_3_bin_0,
hog_cell_7_3_bin_1, hog_cell_7_3_bin_2, hog_cell_7_3_bin_3, hog_cell_7_3_bin_4, hog_cell_7_3_bin_5,
hog_cell_7_3_bin_6, hog_cell_7_3_bin_7, hog_cell_7_3_bin_8, hog_cell_7_4_bin_0, hog_cell_7_4_bin_1,
hog_cell_7_4_bin_2, hog_cell_7_4_bin_3, hog_cell_7_4_bin_4, hog_cell_7_4_bin_5, hog_cell_7_4_bin_6,
hog_cell_7_4_bin_7, hog_cell_7_4_bin_8, hog_cell_7_5_bin_0, hog_cell_7_5_bin_1, hog_cell_7_5_bin_2,
hog_cell_7_5_bin_3, hog_cell_7_5_bin_4, hog_cell_7_5_bin_5, hog_cell_7_5_bin_6, hog_cell_7_5_bin_7,
hog_cell_7_5_bin_8, hog_cell_7_6_bin_0, hog_cell_7_6_bin_1, hog_cell_7_6_bin_2, hog_cell_7_6_bin_3,
hog_cell_7_6_bin_4, hog_cell_7_6_bin_5, hog_cell_7_6_bin_6, hog_cell_7_6_bin_7, hog_cell_7_6_bin_8,
hog_cell_7_7_bin_0, hog_cell_7_7_bin_1, hog_cell_7_7_bin_2, hog_cell_7_7_bin_3, hog_cell_7_7_bin_4,
hog_cell_7_7_bin_5, hog_cell_7_7_bin_6, hog_cell_7_7_bin_7, hog_cell_7_7_bin_8

8. Model 1: Linear Regression

Theory summary: Linear Regression estimates a continuous target value by learning a linear relationship between input features and the output. It is suitable as a clear baseline for regression problems.

Hyperparameters:

```
{'fit_intercept': True, 'positive': False, 'cv_folds': 5, 'learning_rate': 'Not applicable: closed-form ordinary least squares', 'optimizer': 'Not applicable: scikit-learn LinearRegression uses least-squares optimization', 'regularization': 'None', 'batch_size': 'Not applicable', 'epochs': 'Not applicable', 'selected_log1p_transform': False, 'validation_trials': {'without_log_transform': {'MAE': 36757.36525613857, 'MSE': 3831746565.0169415, 'RMSE': 61901.10310016245, 'R2': 0.3153829697065136}, 'with_log_transform': {'MAE': 37912.70246634557, 'MSE': 4419904125.126199, 'RMSE': 66482.35950330131, 'R2': 0.2102970316586037}}}
```

Validation decision on target transform:

```
{'used_log1p': False, 'reason': 'Target skewness was high enough for testing log1p, but validation RMSE did not improve, so the original target scale was kept.', 'train_skewness': 2.740158390971033}
```

Results on test data:

Linear Regression Test Metrics

| Metric | Value |

|---|---|

| MAE | 32,392.65 |

| MSE | 1,975,070,209.27 |

| RMSE | 44,441.76 |

| R2 | 0.4346 |

9. Model 2: KNN Regressor

Theory summary: KNN Regressor predicts a continuous value by finding the nearest training examples and averaging their target values. In this implementation, several k values are evaluated on the validation set, and the k value with the lowest validation RMSE is selected.

Hyperparameters:

```
{'candidate_k_values': [3, 5, 7, 9, 11, 15, 21], 'candidate_weights': ['uniform', 'distance'], 'candidate_metrics': ['euclidean', 'manhattan'], 'cv_folds': 5, 'learning_rate': 'Not applicable: instance-based method', 'optimizer': 'Not applicable', 'regularization': 'Not applicable', 'batch_size': 'Not applicable', 'epochs': 'Not applicable', 'selected_n_neighbors': 11, 'selected_weights': 'uniform', 'selected_metric': 'manhattan', 'selected_log1p_transform': False, 'selection_reason': 'Validation RMSE favored KNN on the original target scale.'}
```

Validation tuning results:

```
{'without_log_transform': {'best_params': {'regressor__metric': 'manhattan', 'regressor__n_neighbors': 11, 'regressor__weights': 'uniform'}, 'best_cv_rmse': 43808.56860151848, 'validation_metrics': {'MAE': 38623.017622751584, 'MSE': 3982731255.5176473, 'RMSE': 63108.88412511861, 'R2': 0.2884065795208759}}, 'with_log_transform': {'best_params': {'regressor__regressor__metric': 'manhattan', 'regressor__regressor__n_neighbors': 15, 'regressor__regressor__weights': 'distance'}, 'best_cv_rmse': 42574.787207500834, 'validation_metrics': {'MAE': 38257.9325491988, 'MSE': 4043240958.4962034, 'RMSE': 63586.48408660604, 'R2': 0.2775953287102584}}}
```

Results on test data:

KNN Regressor Test Metrics

| Metric | Value |

|---|---|

| MAE | 35,912.42 |

| MSE | 2,241,113,673.31 |

| RMSE | 47,340.40 |

| R2 | 0.3584 |

10. Model 3: Logistic Regression

Theory summary: Logistic Regression is a supervised classification model. For multiple classes, it learns decision boundaries that separate the class labels using the extracted image features. A small validation search is used to tune only C, max_iter, class_weight, and optional PCA components.

Hyperparameters:

```
{'penalty': 'l2', 'solver': 'lbfgs', 'multi_class': 'auto', 'candidate_C_values': [0.01, 0.1, 1.0, 10.0], 'candidate_max_iter': [1000, 2000, 3000], 'candidate_class_weight': [None, 'balanced'], 'candidate_pca_components': [None, 120, 180], 'use_light_augmentation': False, 'augmentation_limit': 400, 'learning_rate': 'Controlled internally by lbfgs solver', 'optimizer': 'lbfgs', 'regularization': 'L2 regularization with tuned C value', 'batch_size': 'Not applicable: full-batch deterministic solver', 'epochs': 'Not applicable: max_iter controls solver iterations', 'selected_C': 0.1, 'selected_max_iter': 1000, 'selected_class_weight':
```



```
'balanced', 'selected_pca_n_components': 180, 'augmentation_used': False,
'augmentation_added_samples': 0}
```

Results on test data:

Logistic Regression Test Metrics

```
| Metric | Value |
|---|---|
| Accuracy | 62.65% |
```

Confusion matrix:

```
[[72, 14, 17, 6, 5], [25, 98, 7, 19, 9], [13, 9, 74, 8, 14], [7, 8, 7, 76, 12], [7, 17, 31, 7, 86]]
```

11. Model 4: KMeans

Theory summary: KMeans is an unsupervised clustering algorithm that groups samples into clusters by minimizing within-cluster distance to centroids. Because KMeans does not naturally know class names, this project maps each cluster to a class label using majority voting on the training data.

Hyperparameters:

```
{'candidate_pca_components': [None, 20, 40, 80], 'candidate_n_init': [20, 30], 'init': 'k-means++', 'n_init': 30,
'max_iter': 400, 'algorithm': 'lloyd', 'learning_rate': 'Not applicable: centroid update algorithm', 'optimizer':
'KMeans iterative centroid minimization', 'regularization': 'None', 'batch_size': 'Not applicable', 'epochs': 'Not
applicable: max_iter=300 controls iterations', 'selected_pca_n_components': None, 'selected_n_init': 30,
'n_clusters': 5}
```

Cluster-to-label mapping method:

After fitting KMeans on the training features, each training image receives a cluster ID. For every cluster, the most frequent true class label among the training images in that cluster is assigned as the label for that cluster. Validation and test samples are then classified by predicting their cluster and replacing the cluster ID with the mapped class label.

Cluster-to-label mapping:

```
{0: 'daisy', 1: 'sunflower', 2: 'dandelion', 3: 'tulip', 4: 'dandelion'}
```

Results on test data:

KMeans Test Metrics

```
| Metric | Value |
|---|---|
| Accuracy | 30.71% |
```

Confusion matrix:

```
[[7, 76, 0, 21, 10], [5, 96, 0, 25, 32], [7, 85, 0, 13, 13], [9, 67, 0, 30, 4], [8, 54, 0, 20, 66]]
```

12. Comparison of Models

For regression, Linear Regression and KNN Regressor are compared using MAE, MSE, RMSE, and R2. Lower MAE, MSE, and RMSE indicate better numerical prediction error, while higher R2 indicates stronger explained variance.

For classification, Logistic Regression and KMeans are compared using accuracy and confusion matrices. Accuracy and confusion matrix are classification metrics only. They are not appropriate for regression because regression predicts continuous values rather than discrete class labels.

13. Discussion

The numerical task benefits from categorical encoding because salary datasets usually contain fields such as experience level, employment type, job title, employee residence, company location, and company size. Standardization helps KNN because distance-based models are sensitive to feature scale.

The improved numerical pipeline is stronger because preprocessing, clipping, and target-transform decisions are learned only from the training data. This reduces leakage, makes KNN less sensitive to outliers, and gives Linear Regression a fairer target distribution check without blindly forcing a log transform.

The image task uses compact, explainable features instead of a deep neural network. HOG-style grayscale features capture petal and edge structure, while color histograms and simple image statistics capture color cues that matter for flower classes. This keeps the method classical and discussion-friendly while improving generalization compared with raw pixels alone.

14. Challenges

- Avoiding target leakage in the salary regression task required removing columns that directly describe the same salary value in another format.
- KMeans required a careful cluster-to-label mapping step because it is an unsupervised method.
- Image classification needed a maximum of five classes, and the provided dataset already satisfies this requirement.
- Image features had to be simple enough for academic explanation while still useful for classification.
- KNN and Logistic Regression both required careful scaling so that one feature family would not dominate the distance or optimization process.

15. Conclusion

The project successfully implements four machine learning models across two different data types. Linear Regression and KNN Regressor are used for numerical regression, while Logistic Regression and KMeans are used for image classification and clustering-based classification. The project saves all models, metrics, plots, confusion matrices, and documents in a GitHub-ready structure.

16. References

- Scikit-learn documentation: LinearRegression, KNeighborsRegressor, LogisticRegression, KMeans, train_test_split, StandardScaler, OneHotEncoder.
- Pandas documentation for CSV loading and tabular data handling.
- NumPy documentation for numerical array operations.
- Matplotlib and Seaborn documentation for plotting.

- Pillow documentation for image loading and resizing.